

Mutation Testing

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

NPTEL

Mutation for Integration

- We focus on mutation testing for design integration in two parts:
  - How to use mutation for testing at the integration level without regard to object-oriented relationships.
  - How to use mutation to test for problems involving inheritance, polymorphism and dynamic binding.

NPTEL

Integration Mutation

- Integration mutation (sometimes called interface mutation) works by creating mutants on the connections between components.
- Most of the mutations are around method calls, and both the calling (caller) and the called (callee) method are considered.

NPTEL

Mutation operators for integration mutation

Integration mutation operators do the following:

- Change a calling method by modifying the values that are sent to a called method.
- Change a calling method by modifying the call.
- Change a called method by modifying the values that enter and leave a method.
  - This should include parameters and variables from a higher scope (class level, package, public etc.)
- Change a called method by modifying statements that return from the method.

NPTEL

Interface mutation operators

The following are the five interface mutation operators that we will define.

- Integration Parameter Variable Replacement (IVPR).
- Integration Unary Operator Insertion (IUOI).
- Integration Parameter Exchange (IPEX).
- Integration Method Call Deletion (IMCD).
- Integration Return Expression Modification (IREM).

Integration mutation operator IPVR

- Integration Parameter Variable Replacement (IVPR): Each parameter in a method call is replaced by each other variable of compatible type in the scope of the method call.
- IPVR does not use variables of an incompatible type because they would be syntactically illegal.
- In OO languages, this operator replaces primitive type variables as well as objects.

NPTEL

Integration mutation operator IUOI

- Integration Unary Operator Insertion (IUOI): Each expression in a method call is modified by inserting all possible unary operators in front and behind it.
- The unary operators vary by the language and type.

NPTEL

Integration mutation operator IPEX

- Integration Parameter Exchange (IPEX): Each parameter in a method call is exchanged with each parameter of compatible type in that method call.
- For e.g., if a method call is `max(a,b)`, a mutated method call of `max(b,a)` is created.

NPTEL

Integration mutation operator IMCD

- Integration Method Call Deletion (IMCD): Each method call is deleted. If the method returns a value and it is used in an expression, the method call is replaced with an appropriate constant value.
- In Java, the default values should be used for methods that return values of primitive type. If a method returns an object, the method call should be replaced by a call to `new()` on the appropriate class.

NPTEL

Integration mutation operator IPVR

- Integration Return Expression Modification (IREM): Each expression in each return staement in a method is modified by applying the UOI (Unary Operator Insertion) and AOR (Arithmetic Operator Replacement) operators.

NPTEL

Object-oriented concepts for integration testing

- Several object-oriented concepts and language features are relevant for integration testing of object-oriented languages.
- Some of them apply at the inter-method level (between methods in the same class) and some at the inter-class lecel (between methods in different classes).
- Features that we will re-cap (with testing in mind):
  - Encapsulation
  - Class inheritance
  - Method overriding
  - Variable hiding
  - Class constructors
  - Polymorphism
  - Overloading

NPTEL

Encapsulation

- Encapsulation is an abstraction mechanism to enforce information hiding. It frees clients of an abstraction from unnecessary dependence on design decisions in the implementation of the abstraction.
- Objects can restrict access to their member variables and methods by other objects using encapsulation.
- Java supports 4 such access levels:
  private, protected, public and default (also called package).
- Default access is package.

NPTEL

Encapsulation: Access levels in Java

Specifier	Same class	Different class same package	Different package subclass	Different package non-subclass
private	Y	n	n	n
package	Y	Y	n	n
protected	Y	Y	Y	n
public	Y	Y	Y	Y

NPTEL

Class inheritance

- A subclass inherits variables and methods from its parent and all of its ancestors. Subclass can then
  - use them as defined, or
  - override the methods, or
  - hide the variables.
- Java does not support multiple class inheritance, so every class has only one immediate parent.
- Subclasses can also explicitly use their parent's variables and methods using the keyword `super` (`super.methodname();`).
- Java's inheritance allows method overriding, variable hiding and class constructors.

Method overriding

- Method overriding allows a method in a subclass to have the same name, arguments and result type as a method in its parent.
- Overriding allows subclasses to re-define inherited methods.
  - The child class method has the same signature, but a different implementation.

NPTEL

Variable hiding

- Variable hiding is achieved by defining a variable in a child class that has the same name and type of an inherited variable.
- This has the effect of hiding the inherited variable from the child class.

NPTEL

Class constructors

- A constructor in a class is a special type of subroutine called to create an object.
- It prepares the new object for use, often accepting arguments that the constructor uses to set required member variables.
- They are not inherited in the same way methods are.
- To use a constructor, we must explicitly call it using the `super` keyword. The call must be the first statement in the derived class constructor and the parameter list must match the parameters in the argument list of the parent constructor.

Polymorphism

- Java supports two versions of polymorphism: attributes and methods.
- They both use dynamic binding.
- Each object has a *declared type* and an *actual type*.
  - The actual type can be the declared type or any type that is descended from the declared type.

NPTEL

Polymorphic attribute and polymorphic method

- A polymorphic attribute is an object reference that can take on various types.
- At any location in the program, the type of the object reference can be different in different executions.
- A polymorphic method can accept parameters of different types by having a parameter that is declared of type `Object`.
- Polymorphic methods are used to implement type abstraction.

NPTEL

Overloading

- Overloading is the use of the same name for different constructors or methods in the same class.
- They must have different signatures, or lists of arguments.
- Note: Overloading is different from overriding.
  - The former occurs with two methods in the same class, whereas overriding occurs between a class and one of its descendants.

NPTEL

Instance and class variables in Java

- Instance and class variables are associated with a class.
- Class variables are also called static, in the sense that there is only one copy of the variable that is shared with all instances of that class.
- Instance variables belong to an object (an instance of a class). Every instance of that class has its own copy of the variable. Changes made to the instance variable don't reflect in other instances of that class.
- Levels:
  - Instance variables are declared at class level.
  - Class variables are declared with `static`.
  - Local variables are declared within methods.

Mutation operators for OO features

- Mutation operators are available for integration testing of several OO features:
  - Information hiding language features
  - Inheritance
  - Polymorphism and dynamic binding
  - Method overloading, and
  - classes.
- We list them in the slides that follow.

NPTEL

Mutation operators for OO integration

- Access Modifier Change (AMC): The access level for each instance variable and method is changed to other access levels.
  - The AMC operator helps to generate tests that ensure that accessibility is correct.
  - AMC created mutants can be killed only if the new access level denies access to another class or allows access that causes a name conflict.

NPTEL

Mutation operators for OO integration

- Hiding Variable Delection (HVD): Each declaration of an overriding, or hiding variable is deleted.
  - This causes references to that variable to access the variable defined in the parent (or ancestor), which is a common programming mistake.

NPTEL

Mutation operators for OO integration

- Hiding Variable Insertion (HVD): A declaration is added to hide the declaration of each variable declared in an ancestor.
  - Such mutants can be killed only by test cases that can show that the reference to the overriding variable is incorrect.

NPTEL

Mutation operators for OO integration

- Overriding Method Deletion (OMD): Each entire declaration of an overriding method is deleted.
  - References to the method will then use the parent's version.
  - This ensures that the method invocation is to the intended method.

NPTEL

Mutation operators for OO integration

- Overriding Method Moving (OMM): Each call to an overridden method is moved to the first and last statements of the method and up and down one statement.
  - Overriding methods in child classes often call the original method in the parent class, for e.g., to modify a variable that is private to the parent.
  - A common mistake is to make a call to parent's version at the wrong time. This operator will catch this error.

NPTEL

Mutation operators for OO integration

- Overridden Method Rename (OMR): Renames the parent's versions of methods that are overridden in a sub-class so that the overriding does not affect the parent's method.
- This operator is designed to check if an overridden method causes problems with other methods. For e.g.
  - Two classes: `List` and child class `Stack`.
  - Two methods: `m()` and `f()`, both in class `List`, `m()` calls `f()`.
  - `m()` is inherited without change in `Stack`, but `f()` is overridden in `Stack`.
  - When `m()` is called on an object of type `Stack`, it calls the latter's version `f()` instead of `List`'s version. This may have unintended consequences.

Mutation operators for OO integration

- Super Keyword Deletion (SKD): Delete each occurrence of the `super` keyword.
  - After the change, the reference will be to the local version instead of the ancestor's version.
  - The SKD operator is designed to ensure that hiding/hidden variables and overriding/overridden are used appropriately.

NPTEL

Mutation operators for OO integration

- Parent Constructor Deletion (PCD): Each call to a `super` constructor is deleted.
  - The parent's or ancestor's default constructor will be used.
  - To kill these mutants, we need to find a test case for which the parent's default constructor creates an initial state that is incorrect.

NPTEL

Mutation operators for OO integration

- Actual Type Change (ATC): The actual type of a new object is changed in the `new()` statement.
  - This causes the object reference to refer to an object of a type that is different from the original actual type.
  - The new actual type must be in the same "type family" of the original actual type.

NPTEL

Mutation operators for OO integration

- Declared/Parameter Type Change (DTC/PTC): The declared type of each new object/each parameter object is changed in the declaration.
  - The new declared type must be an ancestor of the original type.
  - The instantiation will still be valid.
  - To kill these mutants, a test case must cause the behavior of the object to be incorrect with the new declared type.

NPTEL

Mutation operators for OO integration

- Reference Type Change (RTC): This right side objects of assignment statements are changed to refer to objects of a compatible type.
  - For e.g., if an `Integer` is assigned to a reference of type `Object`, the assignment may be changed to that of a `String`.
  - Since both are descended from `Object`, both can be assigned interchangeably.

NPTEL

Mutation operators for OO integration

- Overloading Method Change (OMC): For each pair of methods that have the same name, the bodies are interchanged.
  - This ensures that overloaded methods are invoked appropriately.

NPTEL

Mutation operators for OO integration

- Overloading Method Deletion (OMD): Each overloaded method declaration is deleted, one at a time.
  - This operator ensures coverage of overloaded methods— all of them must be invoked at least once.
  - If the mutant still works correctly without the deleted method, there may be an error in invoking one of the overloading methods— the incorrect method may be invoked or an incorrect parameter type conversion has occurred.

NPTEL

Mutation operators for OO integration

- Argument Order Change (AOC): The order of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.
  - This causes a different method to be called, this checking for a common fault in the use of overloading.

NPTEL

Mutation operators for OO integration

- Argument Number Change (ANC): The number of the arguments in method invocations is changed to be the same as that of another overloading method, if one exists.
  - This helps ensure that the programmer did not invoke the wrong method.
  - When new values need to be added, they are the constant default values of primitive types or the result of the default of constructors for objects.

NPTEL

Mutation operators for OO integration

- `this` Keyword Deletion (TKD): Each occurrence of the keyword `this` is deleted.
  - Within a method body, uses of the keyword `this` refers to the current object if the member variable is hidden by a local variable or method parameter that has the same name.
  - This operator checks if the member variables are used correctly by replacing occurrences of `this.X` with `X`.

NPTEL

Mutation operators for OO integration

- Static Modifier Change (SMC): Each instance of the `static` modifier is removed, and the `static` modifier is added to instance variables.
  - This operator validates use of instance and class variables.

NPTEL

Mutation operators for OO integration

- Variable Initialization Deletion (VID): Remove initialization of each member variable.
  - Instance variables can be initialized in the variable declaration and in constructors for the class.
  - This operators removes the initializations so that member variables are initialized to the default values.

NPTEL

Mutation operators for OO integration

- Default Constructor Deletion (DCD): Delete each declaration of default constructor (with no parameters).
  - This ensures that user-defined default constructors are implemented properly.

NPTEL

Mutation operators for integration: Summary

- We have provided an exhaustive list of mutation operators for integration testing of object-oriented applications.
- As for any other mutation, the tester needs to carefully choose only a small subset of these operators, as relevant for testing the concerned application.
- The provided list is meant to be used as a reference to decide on which operators to choose from.

NPTEL